

Exercise 4 — Code Documentation

Exercise Description

Use AI to generate comprehensive documentation for undocumented code. Choose one of the more complex Task Manager algorithms (priority score, text parser, or merge), apply Prompt 1 (Comprehensive Function Documentation) and Prompt 2 (Intent and Logic Explanation), then combine the best of both into a final documented version.

Code chosen: `task_priority.py` (the `calculate_task_score` / `sort_tasks_by_importance` / `get_top_priority_tasks` trio).

Deliverable file: `task_priority_documented.py` — the fully documented version (module docstring + Google-style function docstrings, with inline comments preserved/clarified).

1. Original code

The original functions had only terse one-line docstrings (e.g. `"""Calculate a priority score for a task based on multiple factors."""`) and no description of the scoring model, parameters, return type, edge cases, or examples.

2. Documentation produced with Prompt 1 (Comprehensive Function Documentation)

Applied to `calculate_task_score`, this produced: - **Description** of what the function computes and that it is *pure*. - **Args** with types (`priority: TaskPriority`, `due_date: datetime | None`, etc.). - **Returns** (`int`, can be negative). - **Edge cases:** unknown priority → base 0; `timedelta.days` truncation near day boundaries. - **Example** usage with an expected score.

3. Insights added by Prompt 2 (Intent and Logic Explanation)

Prompt 2 surfaced *intent* that Prompt 1's reference-style docs missed: - The priority weights are **deliberately non-linear** (URGENT emphasised) — worth stating so future maintainers don't "tidy" them into 1/2/3/4. - The scoring factors are **independent and additive**, which is the key mental model → captured as a table in the **module docstring**. - The `key=lambda x: x[0]` in the sort exists to **avoid comparing Task objects** on ties — a non-obvious correctness detail, now documented on `sort_tasks_by_importance`.

4. Final combined documentation

The delivered file merges both: a **module-level docstring** carrying the additive scoring model (Prompt 2's intent), plus **per-function docstrings** with Args/Returns/Notes/Example (Prompt 1's structure). Inline comments were kept but clarified (e.g. noting the non-linear weighting and the truncation caveat). Verified with `python3 -m py_compile` → **syntax OK**.

Submission for Exercise 4 — Code Documentation

1. Original code chosen: `task_priority.py` scoring functions (previously only one-line docstrings).

2. Documentation generated using Prompt 1: comprehensive function docs — description, typed Args, Returns, edge cases, and a worked Example (now in `task_priority_documented.py`).

3. Insights/improvements from Prompt 2: the non-linear priority weighting, the additive-independent scoring model (captured in the module docstring), and the tie-avoiding sort key.

4. Final combined documentation version: `task_priority_documented.py` — module docstring (intent/model) + Google-style function docstrings (structure), syntax-verified.

5. Notes: the hardest thing for the AI to infer was *intent* (why the weights are non-linear) — that needed the domain context from Exercise 3. Prompt 1 nails structure; Prompt 2 is needed for the "why". I'd reuse this two-pass approach (reference docs + intent pass) on my own projects.